



May 19 (Week 1)

N.B.: This research journal is maintained by Asfi Hassan to document the progress, ideas, implementation, and findings arising from research conducted under the supervision of Dr. Ahmed Biniaz at the University of Windsor as part of the NSERC Undergraduate Student Research Award (USRA). This project investigates a computational geometry approach to a problem closely related to the Lonely Runner Problem, a well-known open problem in number theory and discrete mathematics. The purpose of this journal is to provide a clear and organized record of my work throughout the Summer 2026 research term. It is intended to document my understanding, progress, and software development rather than to claim original authorship of all ideas presented. Many of the mathematical concepts and techniques discussed are drawn from existing research and are included to support my learning and to show how they inform my investigations and implementations. Entries may include mathematical observations, algorithmic developments, software design and implementation details, computational experiments, visualizations, meeting notes, and reflections on challenges encountered during the project. This journal serves as a comprehensive record of both the research process and the software tools developed to better understand this problem. Artificial intelligence tools were used to assist in understanding advanced mathematical concepts and research papers, and to support the preparation and refinement of this research journal at an undergraduate computer science level.

Foundational Understanding of the Lonely Runner Problem

Review of Related Literature

The Lonely Runner Conjecture (LRC) is a well-known problem in number theory and combinatorics involving runners moving at distinct constant speeds around a unit circle. The conjecture states that for any set of $n + 1$ runners, each runner will eventually be at least $\frac{1}{n+1}$ units away from every other runner along the circle. By fixing one runner at speed 0, the problem reduces to finding a time t such that the minimum circular distance between the stationary runner and all moving runners is at least $\frac{1}{n+1}$. In this project, the problem is interpreted geometrically by plotting time on the x-axis and the corresponding distance function on the y-axis, producing periodic piecewise-linear graphs whose peaks represent candidate loneliness times. This transformation converts an abstract modular arithmetic problem into a visual and computational geometry problem. The resulting graphs reveal repeating patterns and critical points where the minimum distance function reaches local maxima. These visual structures make it easier to identify candidate witness times and compare the behavior of different speed sets.

Several key papers guided this research. Ram Krishna Pandey proved the conjecture for lacunary speed sets using a constructive interval-based argument, while recent works by Matthieu Rosenfeld and by Touch Sungkawichai and Tanupat Trakulthongchai used computer-assisted methods to verify the conjecture for up to thirteen total runners. These papers show that the problem can be reduced to a finite search over integer speed sets using modular sieves, bounded reductions, and exhaustive computation. Additional recent preprints refine these techniques by combining modular arithmetic, combinatorial reasoning, and algorithmic filtering to eliminate large classes of potential counterexamples. Inspired by these ideas, this project develops an interactive Python framework using Tkinter and Matplotlib to visualize the distance functions and explore the algorithmic structure underlying one of the most significant open problems in modern combinatorics and number theory.

Problem Sketch from Week 1

During the first meeting in week 1 of the research term, I studied a series of introductory sketches provided by my supervisor, Dr. Ahmed Biniiaz, where he explained the core ideas behind the Lonely Runner Problem and several related concepts. These sketches used the unit circle to illustrate how runners move at different constant speeds and showed that a runner is considered lonely when they are at least $1/(k+1)$ units away from every other runner, where $k+1$ is the total number of runners. By fixing one runner with speed 0, the problem becomes easier to analyze geometrically, since the distance between a moving runner and the stationary runner increases up to a maximum of $1/2$ of the unit circle and then decreases symmetrically. This geometric interpretation provides an intuitive way to visualize the problem and forms the basis for representing and studying it using computational geometry techniques within a plane.

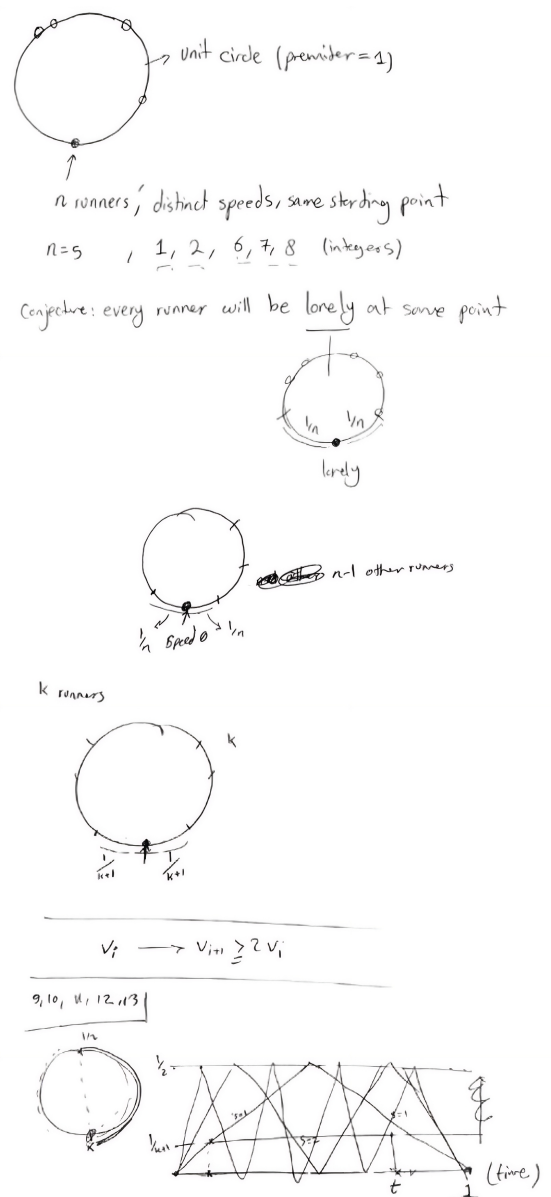


Figure 1: Introductory sketches illustrating the unit circle and graphical interpretation of the Lonely Runner Problem. Sketches provided by Dr. Ahmed Biniiaz.

Current Software Implementation

To support the geometric interpretation of the Lonely Runner Conjecture, an initial Python prototype was developed using the `Tkinter` and `Matplotlib` libraries. `Tkinter` is used to create a graphical user interface (GUI), while `Matplotlib` is used to generate and display mathematical plots directly within the application window. The program launches in full-screen mode and provides two input fields: one for the number of speeds, denoted by n , and another for the corresponding integer velocity values entered as a space-separated list.

The core functionality is implemented in the `draw()` function. After validating the user input, the program iterates through each velocity v and computes a sequence of equally spaced isosceles triangles over the interval $[0, 1]$. Each triangle has base length $\frac{1}{v}$ and height $\frac{1}{2}$, representing one period of the distance-to-nearest-integer function associated with that speed. Since the function $\|tv\|$ is periodic and piecewise linear, the repeated triangles provide an exact geometric representation of the runner's relative distance from the stationary runner over time. Different colors are assigned to distinguish the graphs corresponding to different speeds.

The application also includes a `reset()` function, which clears all input fields and removes the current plot, allowing new speed sets to be entered efficiently. The plotting area is embedded directly into the GUI using `FigureCanvasTkAgg`, and the aspect ratio is fixed so that all triangles retain their correct geometric proportions. This prototype establishes the foundational visualization framework for the project and serves as the basis for future extensions, including the computation of intersection points, automatic identification of candidate loneliness times, and algorithmic verification of specific speed configurations.

The core plotting algorithm iterates through each input velocity v and constructs its corresponding distance function. For each speed, the interval $[0, 1]$ is divided into v equal subintervals of length $\frac{1}{v}$. In each subinterval, the program computes the left endpoint, right endpoint, and midpoint, then draws an isosceles triangle of height $\frac{1}{2}$. Each triangle represents one period of the distance-to-nearest-integer function $\|tv\|$. Repeating this process over all subintervals produces the full piecewise-linear graph for that runner, with distinct colors used to differentiate multiple speeds. The code snippet is provided as a figure below:

```
for idx, v in enumerate(values):
    color = colors[idx % len(colors)]
    triangle_base = line_length / v

    for i in range(v):
        base_left_x = i * triangle_base
        base_right_x = (i + 1) * triangle_base
        top_vertex_x = (base_left_x + base_right_x) / 2

        ax.plot(
            [base_left_x, base_right_x, top_vertex_x, base_left_x],
            [0, 0, y, 0],
            color=color
        )
```

Figure 2: Core algorithm above used to generate the triangular representation of the conjecture.